AMPR EMU · ASSET PACKS

Creating asset packs yourself

Adapted to PS5 Dump & Image Converter – including the AMPR trace assistant, which now handles the recording and the pack profile inside the program itself.

With trace assistant, since v1.9.54 | AMPR_PackTools-4.0, GPL-3.0 | drakmor/ampr_emu, GPL-3.0 | AMPR EMU 0.4.2.1 with LZ4 | zstd with AMPR EMU NG | As of: November 6, 2026

! **A trace never contains all of the game data** – only what was actually read during the recorded runs. Missing levels, languages, or DLC must be added by hand before anything is deleted. This applies unchanged, no matter whether the trace was created by the program or by hand.

i **What differs from the original version of this guide:** It was written for a program called "PS5 Cooling & System Center" that only uploads finished pack files. That does not exist here – **PS5 Dump & Image Converter** performs the trace, the index build, and the profile generation itself (section "Trace assistant") and then packs directly while creating the image (section "Build"). Only a single step remains to be done by hand: checking the generated profile.

01 What an asset pack is – and what it is not

An asset pack replaces the loose game files in the `/app0` folder with one or more seekable, independently compressed LZ4 volumes (`.pak`) plus a directory (`ampr_assets.index`). AMPR EMU reads from it just as it would otherwise read from the loose original files – only faster and with fewer file accesses.

The key point remains unchanged: an asset pack is **not** created by simply “compressing all files”. The vendor recommends building it on a **trace of real read accesses** (`ampr_commands.bin`) from actual play sessions – this yields a proposal (TOML profile) that you should review and complete before packing. It also works without a trace, in which case a cautious, generic profile is used (section “Two ways”).

Source: `drakmor/ampr_emu` , directory `docs/` (`ASSET_PACK_WORKFLOW_EN.md`), GPL-3.0, as well as the tool `AMPR_PackTools-4.0` supplied by the author (included with the program and called by it directly).

02 Two ways to an asset pack

Both ways pack in the end with the same tool and the same safeguards. The only difference is which profile is used as the basis.

RECOMMENDED

With the trace assistant

A tailor-made profile from real play sessions on your own PS5. It identifies more precisely which files are actually needed and provides sensible block sizes based on measured access behavior instead of a blanket assumption.

- Requires a debug build of AMPR EMU (`test-debug-pack`) and time to play through.
- Checks beforehand whether ShadowMount+ really loads this build, switches temporarily if needed, and restores everything at the end.
- The result is placed as `<dump>_ampr_pack.toml` next to the dump and is used automatically the next time you pack.

WITHOUT A TRACE

Default profile

The program packs with a cautious safety default profile (64 KiB, mixed mode, `include=["**"]` with the same protection rules as the trace profile). No additional step needed – just choose the “Asset pack” method and create.

- Works immediately for any title, without playing beforehand.
- Coarser, more generic block and compression choices than a trace-based profile.

Both ways automatically protect system files, executable files, and indexes before packing (`NIE_PACKEN`) – more than the pure vendor route does by itself, which only checks this when the originals are removed later.

03 The trace assistant

Found under **MORE TOOLS → AMPR TRACE**. Completely replaces creating a folder structure by hand, calling an index build script, and starting a separate profile GUI – seven guided steps in one window.

01 **Choose the game dump** – an unpacked, local dump folder (`eboot.bin` / `sce_sys` must be present). Installed titles without their own folder do not work – the trace needs an unpacked game folder on the console.

02 **Connection to the PS5** – as everywhere in the program, via the saved or discovered address.

03 **Match the title on the console** – by title ID. If more than one folder matches, or none matches unambiguously, the program asks which one is meant.

04 **Check ShadowMount+** – reads `/data/shadowmount/config.ini` , detects the version, and determines whether ShadowMount+ would load the recording library at all (box below). If not, the program asks: **Change temporarily** (recommended), **Do not change, record anyway**, or **Cancel**.

05 **Build the index and upload the recording library** – rebuilds `ampr_emu.index` and places it in the game root (`/app0/ampr_emu.index` , **not** in `fakelib/`). The recording-capable AMPR EMU build (`test-debug-pack`) goes into exactly the fakelib folder that ShadowMount+ mounts for this title – as determined in step 4. Whatever is already there is backed up to the computer first; if it cannot be backed up, nothing is

overwritten. If no recording-capable build is available, the program says so and points to the AMPR update.

06 **Record gameplay runs** – five scenarios recommended by the vendor, one after another:

- Cold start through the main menu, load a save
- Load several different levels or large regions
- Fast travel, cutscenes, combat, intensive streaming
- Revisit an area that was already visited
- Additional modes, languages, or installed DLC

After each run: **exit the game normally** (not rest mode), then click "Done playing – fetch trace" in the program. The confirmation is **not modal** – the rest of the program remains usable while you play, even for hours. An incomplete run can be retried, skipped, or the whole assistant can be canceled; runs that already succeeded are kept in that case. Each retrieval opens a fresh connection – after a long play session, ftpsrv closed the old one long ago.

07 **Generate the profile** – combines all successful runs and writes `<dump>_ampr_pack.toml` next to the dump folder, plus a readable report and a metrics file alongside it. If a custom profile already exists there, the program asks before replacing it.

The generated profile is **not** separately "activated" – it simply sits next to the dump and is detected and used automatically the next time this dump is packed (section "Build"). No separate adoption step needed.

STEP 4 IN DETAIL – WHERE THE RECORDING LIBRARY HAS TO GO

ShadowMount+ mounts exactly **one** fakelib folder per title – the first one it finds, even an empty one. A recording library in the wrong folder would remain ineffective, and no trace would ever be created. The program therefore searches in the same order as ShadowMount+:

LOCATION	APPLIES IF …	NOTE
`<Suchpfad>/backports/<TITLE-ID>/`	a backport folder exists for this title – task 7 stores its files there as well. It always takes	The search covers first the game's search path, then the `scanpath` entries of config.ini, and from 1.7beta3 on, finally always

	precedence over the game folder.	`/data/homebrew`. ShadowMount+ up to 1.7 alpha6 reads only `fakelib2` there.
`<Spielordner>/fakelib2` or `/fakelib`	there is no backport folder.	Which one applies depends on the ShadowMount+ version. If both are present, the library goes into both.
newly created	neither of the two folders exists.	`fakelib` from 1.7 alpha8, `fakelib2` up to alpha6. At the end the folder is renamed again (below).

The program then checks whether anything displaces the recording library:

FINDING	CONSEQUENCE	TEMPORARILY
`backport_fakelib=0`	ShadowMount+ mounts no fakelib at all.	`backport_fakelib=1`
`libSceAmpr.sprx` in `/data/shadowmount/emus` and `update_emulators` on (default)	The emulator file replaces the recording library. Simply removing it is not enough – `auto_update_ampr` fetches it again immediately.	`update_emulators=0`
`libSceAmpr.sprx` in the global fakelib and `global_fakelib_priority=global`	The global file wins – unless the title is listed in `global_fakelib_exclude`.	`global_fakelib_priority=g`

Notation from 1.7beta12: The keys in this table apply to ShadowMount+ before 1.7beta12. From 1.7beta12 they are called `[fakelib] enabled` (for `backport_fakelib`), `[emulators] update_enabled` (`update_emulators`) and `[fakelib] global_priority` (`global_fakelib_priority`); `auto_update_ampr` keeps its name and sits under `[emulators]`, `global_fakelib_exclude` is set per game as

`ignore_global_fakelib` in `profiles.ini`. The program detects the format itself (or asks once) and writes the matching names.

From 1.7 alpha8, ShadowMount+ mounts a `fakelib2` on its own, without emulator files and global fakelib – only the main switch matters there. When switching, the program first backs up the previous config.ini to the computer, changes only the keys mentioned, and then reads it back to check that they really are set that way. **The game must not be running during this:** whenever these keys change, ShadowMount+ immediately removes all overlays of a running game.

AT THE END – EVERYTHING IS RESTORED

Also on cancel

After the last step as well as after a cancel, an error, or closing the window – the window closes only when the restoring is finished. If you close the whole program, you are asked; it then waits for the restoring to complete and closes by itself afterwards. Exit the game beforehand for this reason.

Backed-up items come back

A previously existing `libSceAmpr.sprx` and a previously existing `ampr_emu.index` are afterwards back exactly as they were before.

Renamed, not deleted

Where no library was present before, the recording library is afterwards named `libSceAmpr.sprx.mitschnitt`. A folder that did not exist before is renamed in the same way (`fakelib.mitschnitt`) – ShadowMount+ already evaluates an empty fakelib folder and would otherwise give the game the emulator files afterwards. If the name already exists, a number is appended (`.mitschnitt2` …).

ShadowMount+ as before

If the config.ini still looks the way the assistant wrote it, the old file comes back byte for byte, including comments. If someone else has changed it in the meantime, only the assistant's own keys are reset.

Backups are stored under `<dump>_ampr_mitschnitte/vorher_<Datum>_<Uhrzeit>/`, with a separate folder for each run. The trace files in the game folder (`ampr_commands.bin`, `ampr_emu.log`) stay behind, as does a new `ampr_emu.index` where there was none before – without AMPR EMU it does no harm. If the restoring does not succeed completely, for example because the console has been switched off in the meantime, the window and the main log state exactly what still has to be done by hand and where the backups are – when the program exits, additionally in a message before the window closes. A second close during the wait asks whether to quit immediately; the console then stays switched over, and the backups help. As long as the assistant is running, “Shut down the computer after a successful run” does not shut down the computer either.

04 Checking the profile by hand (the most important remaining step)

! **The program does not do this for you.** The vendor explicitly calls this step mandatory ("mandatory"). An automatically generated profile – with or without the trace assistant – is only a starting point. Even a long trace never proves that a file that was never observed is never needed.

The generated `<dump>_ampr_pack.toml` is plain text and can be opened and adjusted in any editor. If you prefer to review it graphically with the familiar overview of "Add files…" / "Add directory…" / "Add pattern…", you can still open the standalone `ampr_pack_gui.exe` from `AMPR_PackTools-4.0`, apply "Load from profile" to exactly this file, make changes, and save it again under the same path – the program simply reads the current file the next time it packs.

Especially important to check: levels/maps/biomes that appeared in no run; every supported language, voice-over, and subtitles; DLC, bonus modes, New Game paths, and final chapters; data for cutscenes, menus, credits, and error handling.

05 Build – as part of the image, not a separate step

Unlike the pure vendor route, there is no separate "Pack and verify" button. Packing happens automatically when, while creating an image (any of tasks 1–6), the **AMPR EMU** checkbox is ticked and **Asset pack** is chosen as the method instead of **Normal** – with the same dump folder, next to which the profile from section 03/04 may be located.

01 If a custom profile is located next to the dump (trace assistant or maintained by hand), it is adopted – the log then explicitly states "Using your own pack profile". Otherwise the default profile from section 02 applies.

02 Pack volumes, `ampr_assets.index`, and the check file `.crc` are built.

03 Each block is verified against the source – **byte for byte**, not just via checksums.

04 A manifest overview checks the actual file, block, and volume counts against the fixed format limits (section “Limits”). If a limit is exceeded, the build aborts – nothing is transferred into the game folder.

05 The verified result (index, runtime file, volumes) goes directly into the same image that this task builds anyway – no separate upload step.

zstd instead of LZ4 (only for ampr-emu-ng): Under **SETTINGS**, the *pack method* can be switched from LZ4 to zstd. This produces `ampr_zstd.index` and `ampr_zstd-NNN.pak`; these packs are read *only* by AMPR EMU NG, while drakmor's runtime does not know them. The profile applies there in the same way: a file `<Ordnername>_ampr_pack.toml` next to the dump, or the rules file chosen in the settings, decides with `compress`, `store`, or `loose` what gets packed – the rule next to the dump takes precedence. Nothing changes for the trace assistant and the LZ4 packs.

COMPRESS, STORE, OR LOOSE – WHAT THE PROFILE MAKES THIS DEPEND ON

ACTION	CHOOSE IF …	MAIN RISK
`compress`	The data can be compressed, or is a mix of compressible and incompressible blocks.	LZ4 costs computing time; a small gain may not justify the decoding.
`store`	The data is already compressed, but bundling it into volumes is still useful (fewer files/handles).	Size barely decreases, accesses still depend on the pack runtime.
`loose`	mmap or another access path that is not intercepted comes into question, the file is rarely needed, or a large file is incompressible without a measured benefit.	Ordinary file system load remains, the original must be kept.

BLOCK SIZE – WHAT THE PROFILE IS BASED ON

BLOCK SIZE	TYPICAL IF …
16 KiB	Traces show very small random accesses to frequently used tables, settings, or localization data.
64 KiB	Unknown or mixed data – the default profile without a trace uses exactly this safe value.
128–256 KiB	Accesses are predominantly large or sequential according to the trace.
512 KiB–1 MiB	Demonstrably sequential streams and already compressed raw media.

Each block costs 12 bytes in the directory that is loaded along with it. For 100 GiB of packed data this amounts to about 75 MiB at 16 KiB, but only about 1.17 MiB at a block size of 1 MiB. If the internal memory pool is not sufficient, the program reduces the buffers by itself and writes this to the log – without repacking.

06 Omitting the originals

During the same build, the program asks – default “No” – whether the original files that have now been packed should be omitted from the result. No separate tool run, no separate GUI needed for this.

Only in a working copy

Nothing is removed directly in the source folder – a working copy must be created for that.

Afterwards verified again byte for byte

Only once every volume has been verified against the source once more are the packed originals removed.

eboot.bin, PRX/SPRX/SELF/ELF, indexes, system folders

Are never removed, even if a faulty profile wrongly listed them as “PACK”.

Only after the console test

The program says so itself in the log: “Removing them is worthwhile only after a successful console test.” The first build should therefore keep the originals.

07 Fixed limits of the format

The program checks these limits automatically against the actual numbers and aborts instead of letting the console reject it.

OBJECT	LIMIT
Files in AMPRPAK4	`2,000,000`
Blocks across all packed files	`16,000,000`
.pak volumes in the entire directory	`1,024`
Pack groups	`256`
Block size	`16 KiB – 1 MiB, power of two`
Requested decoding buffer	`at most 512 MiB`
Requested physical buffer	`at most 128 MiB`

The volume limit applies to the **entire** directory, not per group. Example: 250 GiB with 16 KiB blocks already yields about 16.38 million blocks – before any further files are added. In such a case, a larger block size or deliberately leaving data loose helps.

08 Testing on the console

Check at least: start, new game, several saves, level transitions, fast travel, cutscenes, heavy reloading, language switching, DLC, revisiting an area, canceling a loading process, and exiting normally.

! **For the first tests:** be sure to keep the original files (leave the section “Omitting the originals” at “No”). Remove an original only after all necessary gameplay paths have been tested – and

only for a file that is demonstrably not read via a path that is not intercepted (mmap, direct I/O).

09 Common problems

SYMPTOM	PROBABLE CAUSE	REMEDY
fileId and path do not match	The trace or profile uses an `ampr_emu.index` from a different version of the dump.	Run the trace assistant again for this dump – it always builds index and trace as a matching pair.
No profile available even though traces were made	The assistant was canceled before step 7/7 ran, or no run was complete.	Start the assistant again; individual runs can be skipped without losing the others.
`ampr_commands.bin` is missing after the run – every time	ShadowMount+ did not load the recording library: “Do not change, record anyway” was chosen in step 4/7, or another `libSceAmpr.sprx` wins via a path the program does not know.	Start the assistant again and choose “Change temporarily”. The log shows under “Recording library goes to” where it was placed; with `debug=1`, `/data/shadowmount/debug.log` names the folder that ShadowMount+ really mounted in the line “libraries mounted for”.
“Restore incomplete”	The connection to the console was gone at the end, or ftpsrv rejected a rename.	The message lists each open item individually; the backups are in the named `vorher_…` folder. Catch up by hand with the console's file manager or via FTP.
A volume is missing or fails the check	Build states were mixed, or copying the image was interrupted.	Create the image again – index, runtime file, and volumes always belong together as one set.

A packed file exists, but reading it in the game fails	mmap, direct I/O, or another access path that is not intercepted may be involved.	Test with the originals (section “Omitting the originals”), then add an exact loose rule for this file in the profile (section “Checking the profile”) and rebuild.

10 The most important rules at a glance

- ✓ Traces are **never complete** – add missing levels, languages, and DLC by hand in the generated profile.
- ✓ **Always check** the generated profile – the trace assistant does not replace this step, only the recording and the first version of the profile.
- ✓ First **test completely on the console**, only then think about “Omitting the originals”.
- ✓ While the trace assistant switches ShadowMount+ over or restores it, **the game must not be running** – otherwise ShadowMount+ removes its overlays.
- ✓ eboot.bin, PRX/SPRX files, indexes, and system folders are never packed or removed automatically.
- ✓ Index, runtime file, and all volumes always belong in the image as **one coherent set** – never swap out individual files from it.
- ✓ Task 7 (AMPR EMU Manager) neither builds nor removes asset pack volumes – that happens exclusively when creating an image (tasks 1–6) with the method “Asset pack”.

11 LZ4 and the latest versions (as of October 21, 2026)

Since AMPR EMU 0.4.2.1, asset packs can be compressed with LZ4. According to the vendor's documentation, games become about half the size and load 30 to 40 % faster as a result.

- ✓ **Modes:** `compress` (LZ4, default), `store` (bundled without compression), and `loose` (files stay loose). Blocks that cannot be made smaller are stored unchanged (RAW).

- ✓ **Compression level:** LZ4 HC; high levels take considerably more time when packing, while loading on the console does not become slower as a result.
- ✓ **One set:** index, runtime file, and all volumes belong together and are never swapped out individually.
- ✓ **Error message at runtime** (LZ4/decoding error in the log): the pack is incomplete or damaged – rebuild and verify against the original, do not patch by hand.

Converting directly on the console: exFAT2AMPR

The free console app **exFAT2AMPR** (V-10, GPL-3.0; LZ4 format from DLPSGAME) converts a game into an AMPR LZ4 game directly on the PS5 using the controller – onto internal SSD, M.2 SSD, or USB. No PC needed. Current version: 1.104.

- ✓ Requirements: jailbroken PS5 with kstuff, an ELF loader on port 9021, and ShadowMountPlus from 1.7.
- ✓ Every block is verified, interrupted conversions resume, and the original game is kept.
- ✓ Since 1.104: **startup check** after the conversion (missing or overwritten AMPR emulator, among others), repair via the game menu, time estimate per LZ4 level, and the games' music on the start screen.
- ✓ Keep the original until the converted game has been started once; some games (such as Unity titles) do not run when converted.

This is a different route from the packs of this program: there, the pack is created on the PC while creating an image; here, on the console from a finished game. Both routes use LZ4 and the same AMPR emulator.

Sources: `drakmor/ampr_emu` (release 0.4.2.1, `docs/ASSET_PACK_WORKFLOW_EN.md`) and `exFAT2AMPR/exfat2ampr` (README and releases), retrieved on October 21, 2026.

Compiled from drakmor's own documentation (**ampr_emu**, GPL-3.0) and the behavior of **PS5 Dump & Image Converter** itself. No text is passed on to third parties; this guide is an original, translated summary of the documented procedure for personal use.